

Aurastack AI Core

Aurastack AI Core – Agentic Orchestration Layer (Engineering Brief)

Overview

To build the AI core of the Aurastack platform, your AI Engineer needs to focus on the Agentic Orchestration Layer. This platform does not use a single "Chatbot"; it uses a fleet of Schema-Bound Specialized Agents coordinated by a visual state machine.

1. Core AI Architecture: The "Agentic Unit"

Every agent in the system must follow a strict Input-Process-Output (IPO) architecture. You are not building "prompts"; you are building "functions" wrapped in LLM reasoning.

System Instructions (The Persona)

Each agent needs a specific "Intelligence Profile" (e.g., Senior Underwriter vs. Fraud Analyst).

Input Schema (JSON)

The exact data fields the agent is allowed to "read" from the Global Context.

Output Schema (JSON)

The agent must return a structured JSON object. This is non-negotiable, as the Workflow Engine uses these keys (e.g., risk_score, is_eligible) to branch logic.

Confidence Scoring

Every output must include a confidence_level (0.0 - 1.0). If the score is below a threshold (e.g., 0.8), the system must trigger a Human-in-the-loop (HITL) event.

2. The "Global Context" (Shared Memory)

The AI Engineer must implement a State-Aware Context Object.

- As a case moves through the workflow, each agent appends its findings to this object.
 - **Requirement:** The context must be immutable for past steps but appendable for current steps. This ensures a "Chain of Thought" that can be audited for compliance.
-

3. Tooling & Function Calling

Agents must be equipped with "Tools" (Function Calling). Your engineer needs to build wrappers for:

- **OCR Engine**
For raw document-to-text conversion.
 - **Medical Knowledge Graph**
For cross-referencing CPT/ICD codes.
 - **Search Grounding**
For real-time verification of provider credentials or news.
 - **Database Lookups**
For checking policyholder history in the core insurance system.
-

4. The "Magic Box" (Generative Workflow Engine)

This is a Meta-AI feature. The engineer needs to build a Workflow Generator Agent.

Input

Natural language (e.g., "Build a claims flow for auto glass with a fraud check").

Output

A valid JSON Graph representing the nodes and edges of a workflow.

Logic

The agent must understand the "Insurance Primitives" (Start, AI Task, Decision, Human Task, End) and how to connect them logically.

5. AI Supervisor (Self-Healing & Resilience)

Implement a "Watcher" agent that monitors the execution of other nodes.

Role

If an AI Task fails (e.g., the model returns an invalid JSON or the confidence is too low), the Supervisor agent analyzes the error and selects a Healing Strategy (e.g., "Retry with a more creative temperature" or "Escalate to Human").

6. Agent Directory (The Initial Fleet)

Your engineer should start by implementing these 5 high-priority agents:

- **Doc Intelligence (doc_ext_01)**
High-precision extraction from medical/financial PDFs.
 - **Fraud Sentinel (fraud_det_01)**
Pattern matching against clinical guidelines.
 - **Risk Modeler (risk_model)**
Probabilistic analysis of patient/policy risk.
 - **Intake Orchestrator (intake_ai)**
Intent classification and routing from raw text/email.
 - **Compliance Auditor (audit_ai)**
Scanning execution logs for HIPAA/GDPR deviations.
-

7. Evaluation Framework (LLM-as-a-Judge)

Because this is insurance, "vibe-based" testing isn't enough. The engineer must set up an Evaluation Pipeline:

Ground Truth Datasets

A set of 100+ "Golden Cases" with known correct outputs.

Automated Scoring

Use a "Judge Agent" to score the production agents on:

- Accuracy
 - Schema Adherence
 - Hallucination Rate
-

Summary "We are building a multi-agent system where LLMs are treated as deterministic functions. Every agent is a JSON-in, JSON-out unit. The 'Magic' happens in how the Workflow Engine chains these units together using a shared Global Context."

Aurastack AI Expectations

Aurastack

1. System Health (What you must expose + monitor)

Execution Engine (Core Telemetry)

You need real-time metrics for:

- **Active Workflows**
 - Count of running workflow instances
- **Node Executions**
 - executions/sec or per minute
 - avg latency per node (~ms level)
- **Agent Handoffs**
 - success/failure rate between agents
 - failure reasons (timeout, schema mismatch, tool error)
- **Live Event Bus**
 - streaming logs (Kafka / event stream)
 - must include:
 - agent execution events
 - workflow transitions
 - risk flags

Infrastructure Layer

- **Cluster Registry**
 - region
 - status (online / maintenance)
 - load %
- **Resource Metrics**
 - CPU
 - Memory
 - Network IO
 - Disk
- **Versioning**
 - Kernel version
 - Orchestrator version

Engineering implication:

You need:

- observability stack (Prometheus + Grafana or equivalent)
 - event streaming (Kafka / NATS)
 - distributed tracing (OpenTelemetry)
-

2. Audit Trail (Non-negotiable system design)

Core Requirement: WORM Logging

Every event must be:

- append-only
 - immutable
 - cryptographically signed
-

Log Schema (Must Implement)

Each log entry:

```
{
  "timestamp": "...",
  "event_type": "CASE_CREATED",
  "details": "...",
  "actor": "agent/user/system",
  "case_id": "...",
  "signature": "hash",
  "verification_status": "valid"
}
```

Required Features

- Search (by case_id, event_type, actor)
- Export (compliance reports)
- Log drill-down (full context)

Engineering implication:

- Use hash chaining or Merkle tree
 - Store in immutable storage (S3 + versioning / append DB)
 - Signature verification pipeline
-

3. Access Control (RBAC – enforce everywhere)

Core Model

- Role → Permissions → Resources
-

Required Roles (baseline)

- ADMIN
 - ADJUSTER
 - AUDITOR
-

Permission System

Must support granular permissions like:

- VIEW_MEDICAL_RECORDS
 - EDIT_FINANCIALS
 - MANAGE_ROLES
 - VIEW_AUDIT_TRAIL
 - EDIT_WORKFLOWS
 - IMPORT_DATA
-

Enforcement Layer

Every request must pass:

Auth → Role → Permission Check → Resource Access

Compliance Mapping

Roles must map to:

- HIPAA
- GDPR
- SOC2

Engineering implication:

- Middleware-level enforcement (not UI-level)
 - Policy engine (OPA recommended)
 - Field-level masking support
-

4. Compliance Engine (What must be trackable)

You need system-level compliance states for:

- SOC2 Type II
 - GDPR
 - ISO 27001
 - HIPAA
-

Each Standard Must Track

- Last audit date
- Compliance score (0–100%)

This is NOT static → must be computed from:

- audit logs
 - access violations
 - data handling events
-

5. Data Governance (Hard constraints for agents + workflows)

1. Immutable Audit Trail

- Already covered → enforce globally

2. Data Minimization (Critical for agents)

Agents MUST:

- only receive required fields (input schema enforced)
- never access full case context unless needed

Engineering implication:

- schema-based context injection
 - no raw DB access for agents
-

3. Encryption

- **At Rest:** AES-256
 - **In Transit:** TLS 1.3
-

4. Zero-Trust Architecture

Every call (even internal):

Service → Auth → RBAC → Execute

Includes:

- agent → tool calls
 - workflow → API calls
 - internal microservices
-

What Needs to be Build

Core Systems

1. **Execution Engine**

- workflow runtime
 - agent orchestration
 - event-driven
 - 2. **Observability Stack**
 - metrics + logs + tracing
 - 3. **Audit System (WORM)**
 - append-only logs
 - cryptographic signatures
 - 4. **RBAC Engine**
 - role-permission mapping
 - enforcement middleware
 - 5. **Agent Sandbox**
 - schema-bound inputs/outputs
 - restricted tool access
 - 6. **Compliance Layer**
 - audit scoring
 - violation detection
-

Non-Negotiable Constraints

- No agent runs without schema
- No workflow runs without logging
- No API call without RBAC check
- No data access without audit trail
- No decision without traceability